

INDUSTRY INSIDER TALK · VNU HCMUS

HOW GENERATIVE AI IS REDEFINING SOFTWARE TESTING

An evolution story — from a single prompt to an autonomous quality platform

Nguyen Duc Viet (VietND96) · Senior Software Engineer in Test @ HRS Group

ABOUT THE SPEAKER

ABOUT ME

Nguyen Duc Viet

VietND96

Senior Software Engineer in Test
@ HRS Group

Ho Chi Minh City, Vietnam

github.com/VietND96 · linkedin.com/in/vietnd96

WHAT I DO

I build software-defined test automation — modern test automation frameworks and AI-driven testing workflows — and I contribute to the Selenium open-source project.

Focus: test automation · CI/CD · AI in testing

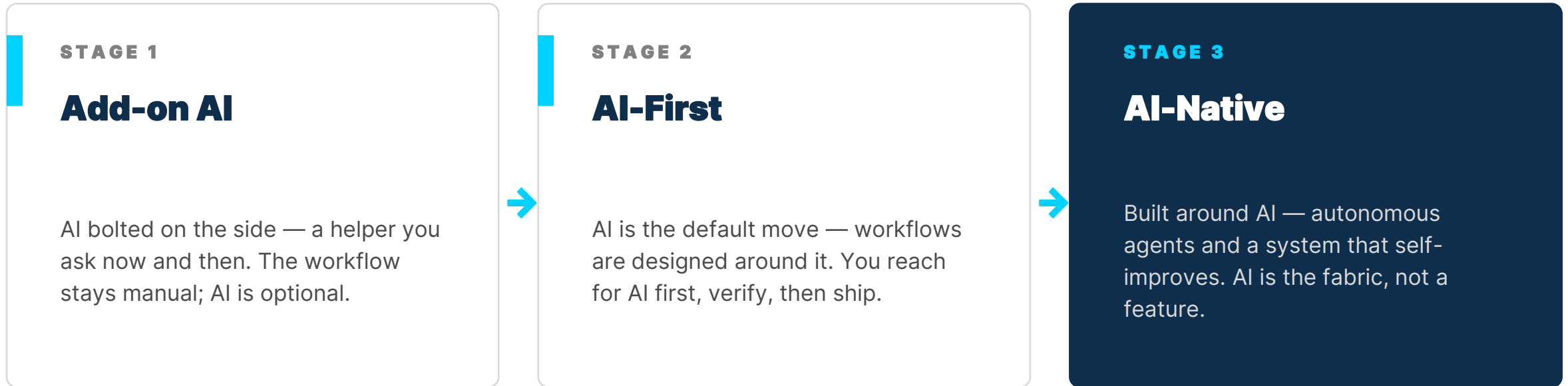
HOW AI CAPABILITY GREW — CHAPTER BY CHAPTER

- 1 Generative AI in software testing — the awareness
- 2 Prompt engineering for QA
- 3 Modern framework enabling AI agents (MCP · CLI · Skills)
- 4 Autonomous testing — agent & multi-agent workflows
- 5 CI/CD & AI — failure analysis & self-healing
- 6 The cross-team quality-intelligence platform
- 7 Skills & career path in the AI / SDET era

Every example is from real, running code — not marketing slides.

GENAI ADOPTION: ADD-ON → AI-FIRST → AI-NATIVE

Adopting GenAI in testing is a maturity curve, not a switch you flip.



The real shift: AI goes from a tool you occasionally use → to the system your testing is built on.

WHAT CHANGES AS YOU CLIMB THE CURVE

As adoption matures, AI takes on more of the doing — and the human moves from operator to supervisor.

	ADD-ON AI	AI-FIRST	AI-NATIVE
AI's role	Assistant, on request	Default drafter	Autonomous operator
Human's role	Does the work	Directs & verifies	Supervises & governs
In testing	Draft a test case from a requirement	Generate + classify in the pipeline	Agents plan → generate → heal

You are here → most teams sit between Add-on and AI-First; the rest of this talk climbs toward AI-Native.

USING GENAI LIKE A PRO — ACROSS THE QA LIFECYCLE

1 Understand

Flag ambiguous requirements & missing acceptance criteria.

2 Design

Derive edge cases, boundary / equivalence, decision tables.

3 Data

Synthetic + boundary + PII-safe test data.

4 Automate

Turn a manual case into Playwright code.

5 Triage

Classify a failure: real bug vs flaky vs env · summarize logs.

6 Report

Draft a release-readiness summary for stakeholders.

At every stage: **AI drafts, the pro QA judges.** Amplify judgment — never replace it.

ELEMENTS OF A GOOD PROMPT

A GOOD PROMPT HAS

- 1 Persona** — act as a senior QA expert
- 2 Context** — the requirement & acceptance criteria
- 3 Task** — generate test cases
- 4 Coverage** — happy · edge · error paths
- 5 Output format** — Excel table + exact columns
- 6 Constraints** — one case / row · atomic · traceable

EXAMPLE — TEST CASES → EXCEL

You are a senior QA expert.

Requirement: {paste the user story + acceptance criteria}

Write test cases covering the
happy path, edge cases, and error paths.

Return an Excel-ready table with columns:

ID | Title | Preconditions | Steps | Test Data |
Expected Result | Type (happy/edge/error) | Priority

Rules: one case per row · atomic steps ·
map each case to an acceptance criterion.

PROMPT VS SKILL

The next step up: package a good prompt into a reusable Skill.

FEATURE	PROMPT	SKILL
What it is	What you type	What you build
Lifespan	One session	Reusable forever
Who can use it	You, this time	Your team, every time
Result quality	Depends on the typer	Same every run

A skill is a prompt, packaged — **versioned, shared, repeatable**. Here's what a Skill actually looks like →

WHAT A SKILL LOOKS LIKE: SKILL.md

A FOLDER + ONE SKILL.md

```
my-skill/
├ SKILL.md      name + description + steps
├ scripts/      code the agent can run
├ references/    docs, loaded on demand
└ assets/       templates & files
```

SKILL.md

```
---
name: pr-review
description: when & how to review a PR
---

# Instructions (Markdown)
steps • checklists • examples
```

ONE OPEN STANDARD

Agent Skills spec (agentskills.io) — open-sourced Dec 2025, originally from Anthropic; now a cross-agent format.

WORKS IN 30+ AGENTS TODAY

Claude Code • Codex CLI • Gemini CLI • GitHub Copilot • Cursor • Windsurf • OpenCode • Cline • VS Code ...

HOW IT LOADS — PROGRESSIVE DISCLOSURE

Only name + description sit in context at first. The full SKILL.md loads when a task matches — a big library, a small footprint.

BUILD OR CHOOSE A FRAMEWORK

BROWSER LIBRARY

Selenium

A browser-automation library — you build the framework around it: runner, fixtures, reporting.

E2E FRAMEWORK

Cypress

End-to-end framework out of the box: runner, assertions, retries.

E2E FRAMEWORK

Playwright ✓

End-to-end framework out of the box — fast, cross-browser, auto-wait.

Selenium = a library you wrap in a framework. Cypress / Playwright = frameworks you extend.

Then build your suite on top: one monorepo — many suites on shared libraries.

Web UI

Mobile

API

Monitoring

Product suites — one per application under test

Shared libraries — imported by every suite:

Config & env

Core: fixtures · auth · pooling

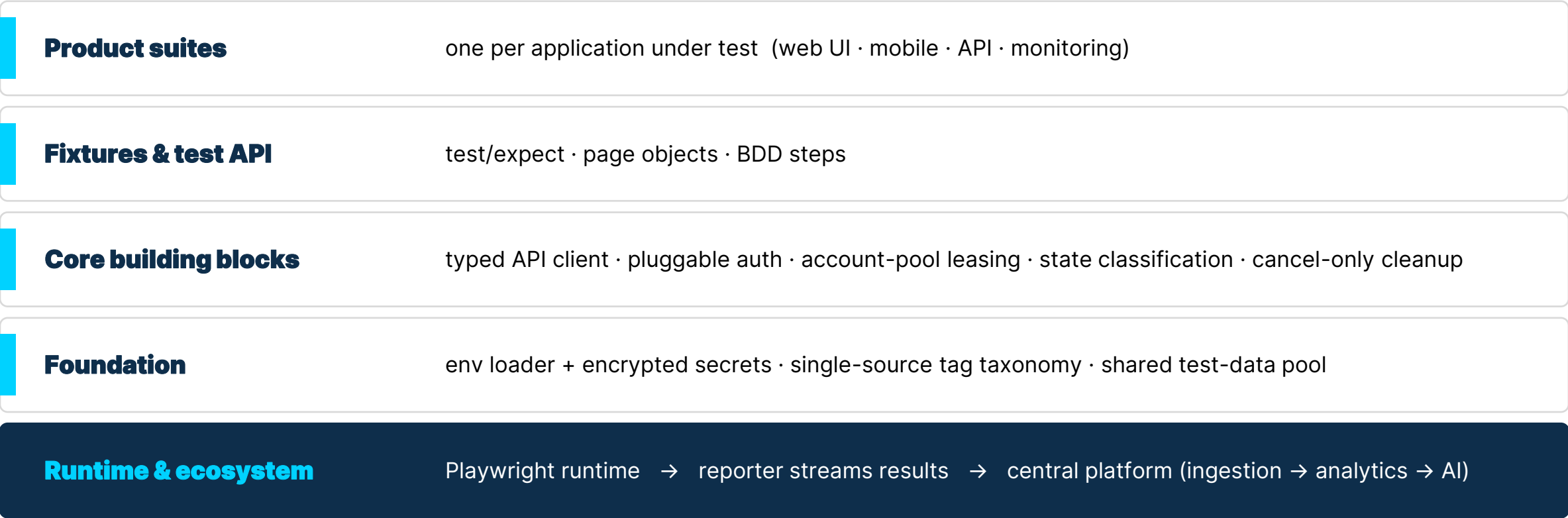
Test data

Reporting

One root install · no per-suite lockfiles · one consistent version set.

HIGH-LEVEL DESIGN — A LAYERED APPROACH

Abstract view: layered by responsibility — each layer uses only the one below; products never depend on each other.



Approach: shared abstractions + one composition seam per concern = reuse by default, and the suite scales to many teams.

A MODERN FRAMEWORK = TOOLS + DESIGN + ECOSYSTEM

1 Modern tools

- Playwright · TypeScript · ESM
- npm-workspaces monorepo
- One root install, no per-suite lockfiles

2 Design: scale & reuse

- Shared libraries every suite imports
- Single-source tag taxonomy → selection + projects
- Pluggable auth + account-pool leasing
- Safe by construction: cancel-only cleanup + lint guards

3 Ecosystem = the platform

- Reporter streams results → ingestion
- Analytics: Test Impact, flakiness
- AI & agents: classify, self-heal
- The framework is not an island

A modern automation framework isn't just a tool — it's **modern tools + design for scalability & reuse + an ecosystem around it (the platform)**.

HOW PLAYWRIGHT LETS AI AGENTS WRITE & DEBUG TESTS

A modern framework doesn't just run tests — it exposes the seams an AI agent needs to generate and fix them.

Playwright MCP

Agents drive a real browser live via the Model Context Protocol — using the accessibility tree, not screenshots. Structured, reliable, cheap to reason over.

Playwright CLI + Skills

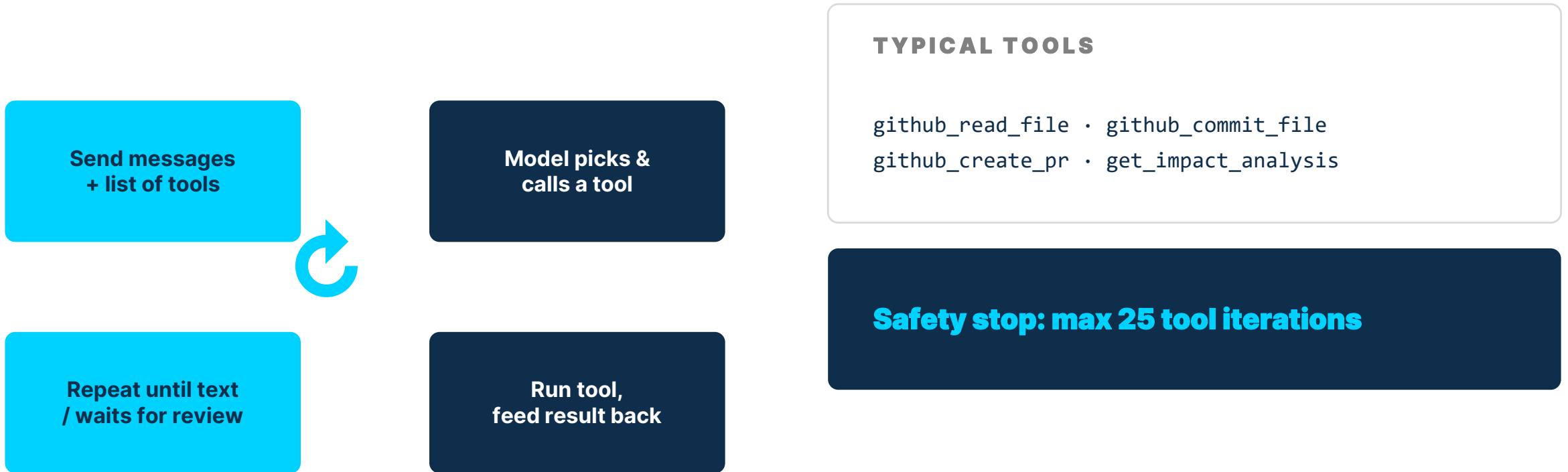
A token-efficient CLI for coding agents: snapshot → click → screenshot, state saved to disk, not streamed into context. $\approx 4\times$ fewer tokens.

AI Skills (SKILL.md)

A folder + SKILL.md teaches the agent how to use the CLI / framework. Loaded only when a task needs it — progressive disclosure.

MCP = act in a live browser · CLI + Skills = act cheaply at scale · **both feed the agent real page context to generate and debug tests.**

AN “AGENT” IS REALLY A TOOL-CALLING LOOP



Not magic — the inside is surprisingly simple. Understand this loop and you understand most “agentic AI.”

SAFE PAUSING & AN AGENT THAT REVIEWS PRs

HUMAN-IN-THE-LOOP, DONE SIMPLY

__AWAITING_REVIEW__
__AWAITING_INPUT__

A tool returns a sentinel string → the run pauses and the whole conversation is checkpointed to Redis → it resumes exactly where it stopped once a human approves or replies.

Long-running agents must pause and save state — not run in one shot.

AN AGENT THAT REVIEWS THE PR

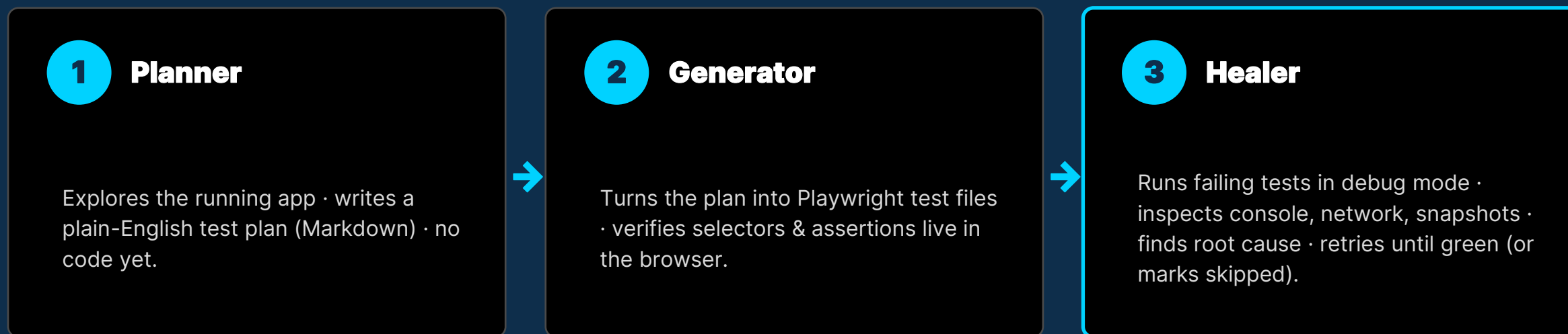
When a pull request arrives:

- 1 Fetch the PR's changed files
- 2 Run Test Impact Analysis → find affected tests
- 3 Post a structured review comment on the PR

It acts like a first-pass reviewer.

PART 4 · AUTONOMOUS TESTING

MULTI-AGENT WORKFLOW: PLAN → GENERATE → HEAL

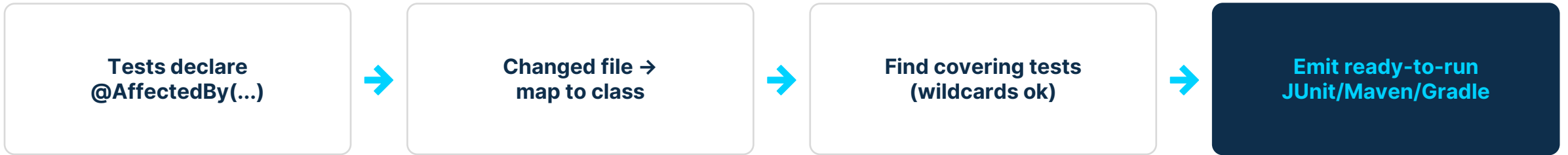


`npx playwright init-agents --loop=claude | vscode | opencode` · all built on MCP

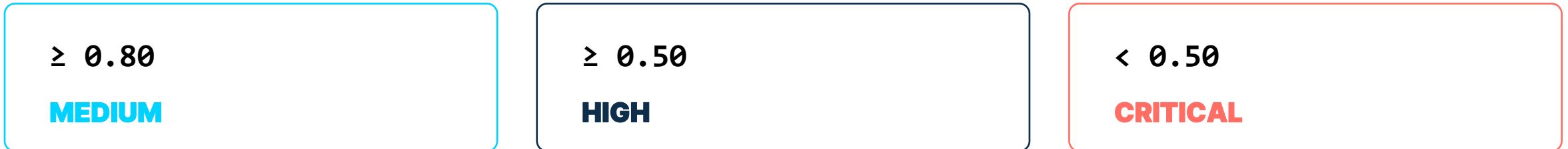
Generate, then debug — a multi-agent workflow. **AI writes; humans still review the PR.**

TEST IMPACT ANALYSIS: RUN ONLY THE TESTS THAT MATTER

Which files changed decides which tests run — instead of running the whole suite on every commit.



A risk model, based on how much of the changed code is covered:



CRITICAL → fall back to the full suite. Ready-made templates for GitHub Actions · Azure DevOps · Jenkins.

FAILURE ANALYSIS → A TEST THAT HEALS ITSELF (5 STEPS)

1

Gather context

10 real failure samples (30 days) + AI failure analysis + flakiness score.

2

Read the test

The AI calls a tool to read the broken test's source code.

3

Fix it

Wrong selector · missing wait · bad assertion → commit to a branch.

4

Open a DRAFT PR

"Always created as draft until a human approves."

5

Request review

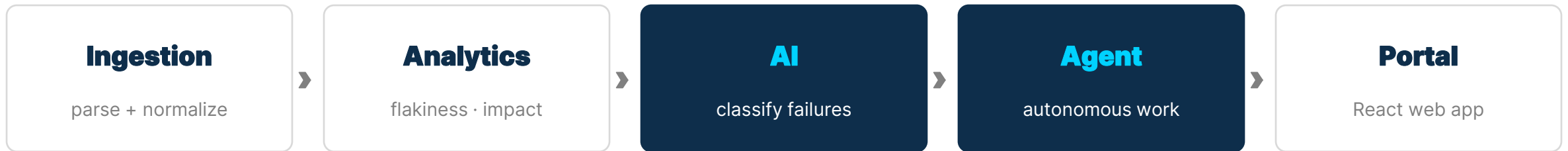
Pause and wait for a person to sign off before it's un-drafted.

AI does the repetitive work · humans keep the decision to merge

PART 6 · QUALITY-INTELLIGENCE PLATFORM

WHERE IT ALL COMES TOGETHER: ONE PLATFORM

Everything so far — generation, prompts, agents, self-healing — centralized for every team, in one place.



Data flows one way through Kafka · PostgreSQL is the single source of truth · each service owns one slice.

A CROSS-TEAM QUALITY BACKBONE

Ingest any results → compute signals → classify failures → run agent workflows — **org-wide**.

PART 6 · QUALITY-INTELLIGENCE PLATFORM

HIGH LEVEL — AND WHERE IT SITS

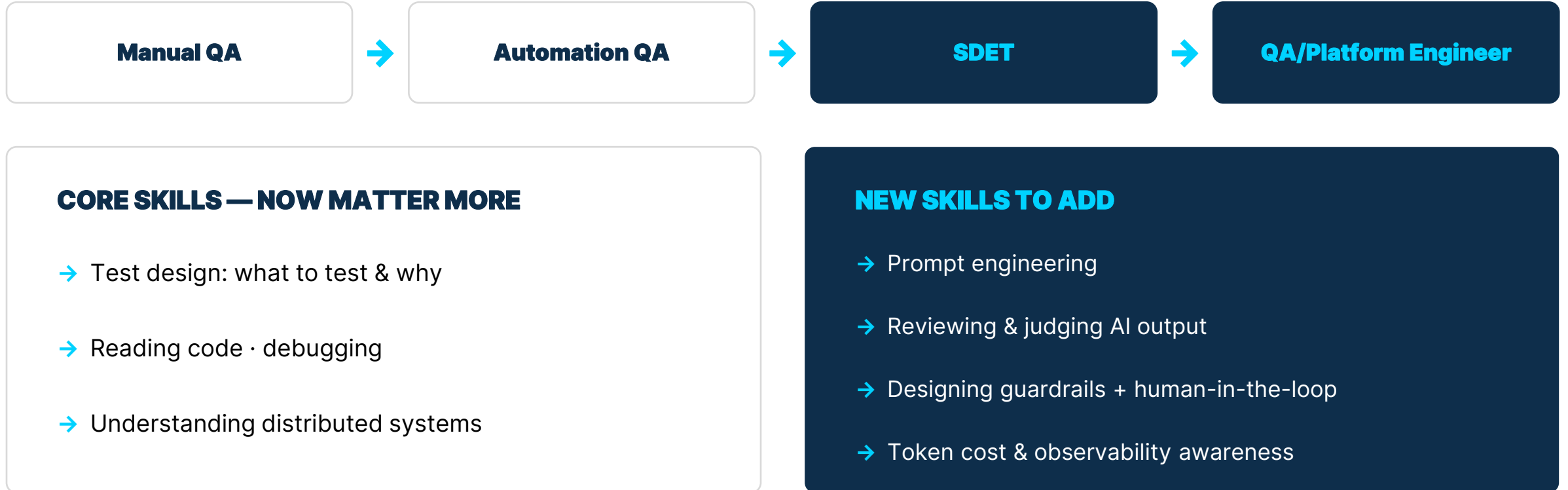
One central brain for quality — between your test toolchain and your teams.



Its position: a central layer that turns every test run into org-wide quality signals — one platform, not per-team silos.

Refer to how test platform implementation: <https://github.com/NDViet/test-automation-platform>

QA/SDET CAREER PATHS IN THE AI ERA



Do 3 things now: (1) adopt AI in how you work · (2) build a small but valuable REAL project · (3) think about scale.

ONE LINE TO TAKE HOME

QA ISN'T GOING AWAY — IT'S LEVELING UP.

AI doesn't replace great QA/SDET — it makes the ones who master it indispensable.

Let the machine do the repetitive work · let humans keep judgment and the decisions.

THANK YOU • HAPPY TESTING

Ask me anything: the craft, the career, where to start.

Nguyen Duc Viet (VietND96) • github.com/VietND96 • linkedin.com/in/vietnd96